Namma Space: Smartphone Capture to Photorealistic Browser Walkthrough

Techfest IIT Bombay 2026–27 · Round 1: 3D Reconstruction & Roaming · Team: [TEAM NAME / ID] · Live viewer: [PUBLIC URL] · Code: [REPO URL]

> **DRAFT, NOT FINAL.** Orange items are placeholders for values that have not been measured yet, because the real-room capture has not been processed. Do not submit this version.

1. Problem and proposed solution

Static 2D floor plans do not convey what an indoor space looks or feels like, and building 3D models manually does not scale. Namma Space is an end-to-end, software-only pipeline. A non-expert records one walkthrough video of a room on a smartphone following a fixed SOP. The pipeline then reconstructs a photorealistic **3D Gaussian Splatting (3DGS)** digital twin with no manual modelling and publishes it as a zero-install web page. Visitors can roam it in first person with **WASD + mouse**, like a video game.

Round 1 delivers capture, reconstruction and free roaming. The viewer is built on three.js, so Round 2 overlays (POIs, floor paths) attach to the same scene graph.

2. System architecture

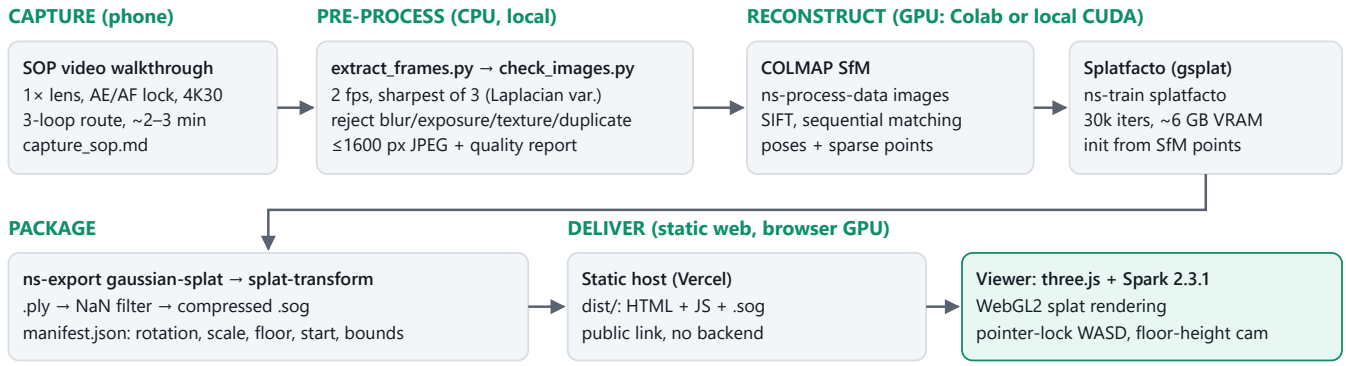


Figure 1. Pipeline. Each stage is one command in `scripts/run_reconstruction.ps1` / `.sh`, which logs per-stage timings to `run_metrics.json`.

3. Capture SOP and reconstruction methodology

3.1 Capture SOP (summary of `capture_sop.md`)

- **Space:** one furnished room, 10–30 m², even lighting. Remove moving objects and private items. Cover mirrors.
- **Phone:** main 1× lens only. Video at 4K30. AE/AF locked. No zoom and no "super-steady" mode, so the camera keeps one intrinsic model.
- **Motion:** about 0.3 m/s, always translating while turning (parallax for SfM), chest height, tilted 10–15° down, 1–3 m from surfaces.
- **Route:** entrance view, then an outer loop facing the centre with 3–5 s corner pauses, then an inner loop facing the walls, then furniture arcs at two heights, then back to the entrance (loop closure).
- **Failure prevention:** a table maps blur, reflections, textureless walls, moving objects and exposure drift to their symptoms and fixes.

3.2 Processing

1. **Frame selection:** decode at 6 fps and keep the frame with the highest variance of Laplacian in each group of 3. This gives 2 fps with even spacing and less motion blur.
2. **Quality gate:** reject a frame if sharpness < max(40, 0.35 × median), brightness falls outside 35–225, clipped pixels exceed 25 %, fewer than 15 % of 32-px blocks are textured, or its 16×16 dHash is within 6 bits of the last kept frame.
3. **SfM:** COLMAP via `ns-process-data`, with sequential matching for video and exhaustive matching for photo sets. Accept the run if registered ÷ used ≥ 0.8.
4. **3DGS training:** `ns-train splatfacto`, 30 k iterations, Gaussians initialised from SfM points.
5. **Export and compression:** `ns-export gaussian-splat` → PLY, then `splat-transform --filter-nan` → SOG.

6. **Alignment:** the viewer's `?calibrate` panel levels the floor (z-up → y-up), sets metric scale from a known dimension, and estimates floor height and walk bounds from 2nd/98th-percentile splat centres.

3.3 Our contributions (beyond the open-source tools)

COLMAP, Splatfacto and Spark do the heavy lifting. What we built is the part that makes them usable by a non-expert and reproducible:

- The **capture SOP** and its failure-mode table.
- **Sharpness-based frame selection** and a **fail-fast quality gate**.
- **Pipeline automation** with isolated run folders, a COLMAP registration gate, fresh-output checks and automatic metrics.
- **Offline calibration estimates** (floor, bounds, start pose) written to a validated draft manifest.
- An in-browser **calibration tool** with a floor-outline editor.
- The **first-person viewer integration** with robust loading and error handling.

4. Technology choices

Layer	Choice	Justification (alternatives considered)
Representation	3D Gaussian Splatting [1]	Photoreal, view-dependent appearance at real-time frame rates in WebGL. NeRF needs ray marching per pixel, which is too slow for the browser. Meshes (Meshroom) lose thin and glossy detail.
Poses	COLMAP [2]	Reference incremental SfM, scriptable, and provides initial points. DUSt3R [4] is kept as a fallback for sparse or low-texture captures.
Training	nerfstudio Splatfacto / gsplat [5,6]	One CLI for processing, training and export. About 6 GB of VRAM fits a free Colab T4. Apache-2.0, unlike the non-commercial original 3DGS code.
Web renderer	Spark 2.3.1 + three.js 0.186	A splat mesh is a normal three.js object, so custom FPS controls and future POI and path overlays work. Loads PLY, SPZ and SOG. MIT licence.
Compute	Hosted GPU (Colab) by default	The local RTX 5060 (Blackwell, sm_120) needs CUDA ≥ 12.8 and MSVC for the gsplat JIT build. Colab avoids losing the deadline to installation. A local path is documented as well.

5. Reconstruction results (PENDING)

PENDING: entrance view of the real reconstructed room

PENDING: corner view showing wall and furniture detail

PENDING: close-up of a textured object plus a weak region (e.g. window)

Figure 2. Screenshots will be taken from the deployed viewer and the real model only.

6. Web viewer and interaction design

- **Start overlay** with a key legend. The Start button is enabled only once the model has decoded. The progress bar shows downloaded MB and percentage.
- **Movement:** pointer-lock mouse-look (pitch clamped to ±85°) and WASD/arrow keys at 1.4 m/s (Shift 2×). Exponential velocity smoothing gives fluid starts and stops. The camera is held at floor + eye height (1.6 m, Q/E ±0.6 m).

PENDING: viewer screenshot on the deployed URL with the real model (overlay + walking view with HUD)

- **Navigation bound:** the camera stays inside a calibrated floor outline (point-in-polygon test with wall sliding), or a rectangle if no outline is set. This is explicitly *not* collision detection; splats have no surfaces.
- **Controls and robustness:** R reset, Esc pause and resume, an exit button, fullscreen, and a HUD (FPS, splats, size, load time, position). The manifest is validated field by field before loading. The download streams into a single preallocated buffer and has a stall timeout. Clear error cards cover an invalid or missing manifest or model, HTML 404 fallbacks, no WebGL2, and GPU context loss. Built for a desktop with keyboard and mouse; touch screens get drag-to-look and a notice.

7. Measured results and testing methodology

Metric (real room)	Value	Metric (real room)	Value
Video duration / resolution	PENDING	Training time / GPU	PENDING
Frames kept / rejected	PENDING	Splat count / PLY size	PENDING
Registered cameras / used	PENDING	SOG size (web)	PENDING
COLMAP time	PENDING	Load time / FPS (deployed)	PENDING

Testing performed on test data (2026-10-27/29):

- **Automated tests** (`npm test`, also in GitHub Actions CI with the production build): 19 JS tests for manifest validation and an L-shaped walk outline, and 5 Python tests for frame extraction, the quality gate (pass, fail and zero usable frames) and splat statistics with the draft manifest.
- **Pipeline gates on Windows:** each gate and failure path stops the run with a clear message.
- **Synthetic room:** 79,102 splats in nerfstudio's z-up PLY layout, converted to a 155,108-byte SOG. The draft manifest recovered its floor (0.0 m) and bounds exactly.
- **Viewer:** holding W for 1.5 s moved the camera 2.11 m (expected about 2.1 m). Invalid or missing manifests and models showed the correct errors.

Planned on the real model: cold-cache load time, peak memory and average and minimum FPS over a 60 s walk in incognito on two browsers, plus a pointer-lock, Esc and resume check.

8. Limitations, reproducibility and roadmap

Limitations

- No physical collision. You can walk through furniture; only the footprint bound applies.
- COLMAP is scale-free, so metric scale is set manually from a known dimension.
- Textureless walls, mirrors and windows can produce floaters or holes. The SOP mitigates this, but splats do not fully solve it.
- Large models require compression and possibly external hosting.

Reproducibility

Exact version pins (npm lockfile, nerfstudio 1.1.5), one script per platform plus a Colab notebook, metrics written

Roadmap

- **Round 2:** a floor occupancy grid from splat centres and COLMAP points (height slice), A* paths drawn as a floor ribbon, collision against the same grid, POIs stored in the manifest with click-to-label (raycast on the splat mesh) and search-to-jump.
- **Quality:** depth and normal priors (DN-Splatter) and line and plane cues (IndoorGS [3]) for flat indoor walls.
- **Scale:** multi-room capture with doorway linking passes and vocab-tree matching.
- **Round 3 bonus:** WebXR AR navigation, localising the phone against the SfM map.

automatically, and sample data with provenance in `test/`.

9. References and acknowledgements

1. B. Kerbl, G. Kopanas, T. Leimkühler, G. Drettakis. 3D Gaussian Splatting for Real-Time Radiance Field Rendering. ACM TOG (SIGGRAPH) 42(4), 2023.
2. J. L. Schönberger, J.-M. Frahm. Structure-from-Motion Revisited. CVPR 2016.
3. C. Ruan, Y. Wang, T. Guan, B. Zhang, L. Ju. IndoorGS: Geometric Cues Guided Gaussian Splatting for Indoor Scene Reconstruction. CVPR 2025.
4. S. Wang, V. Leroy, Y. Cabon, B. Chidlovskii, J. Revaud. DUSt3R: Geometric 3D Vision Made Easy. CVPR 2024.
5. V. Ye et al. gsplat: An Open-Source Library for Gaussian Splatting. JMLR 26, 2025.
6. M. Tancik et al. Nerfstudio: A Modular Framework for Neural Radiance Field Development. SIGGRAPH 2023.

Third-party software (used under licence, credited per rule 4): COLMAP (BSD-3), nerfstudio 1.1.5 and gsplat 1.4.0 (Apache-2.0), Spark 2.3.1, three.js 0.186.1, splat-transform 3.10.1 and Vite 8.3.4 (MIT), FFmpeg via imageio-ffmpeg (LGPL/GPL binary, BSD-2 wrapper), NumPy and Pillow, and Inter and JetBrains Mono fonts (OFL). Full list in `references.md`.